CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 15

StudentHub – Final Integrated Campus Portal

PHP 8 · MariaDB · HTML5 / CSS3 / JavaScript · Fetch API · sessions, RBAC and CSRF · Python + Playwright tests

Report – implemented in the integrated StudentHub portal (studenthub/)

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

New report. No Word or PDF report existed for Practical 15. Practical 15 is the final integrated portal: Practicals 5–14 were integrated step by step into the one canonical application `studenthub/` (one application, one database `studenthub_db`, one login), and `Practical-15/` is now a verified byte-for-byte snapshot of it (`Practical-15/README.md`, `Practical-15/SNAPSHOT.md`, `tools/sync_practical15.py`). This report covers the architecture, the database (migrations 001–005), where each practical lives in the portal, installation, configuration and deployment, the snapshot process, the complete test run and the end-to-end scenario. All screenshots were taken from the canonical application `studenthub/` with Microsoft Edge (headless, Playwright) on disposable test databases with fictional records; all test results are copied from the saved test outputs of the repository. The commit that records the snapshot is not named here – see the repository history.

1. Problem Definition and Objectives

Topic (root `README.md`, practical status table): *Integrated portal (copy of studenthub/)*; title of `Practical-15/README.md`: *StudentHub – Final Integrated Campus Portal*.

Practicals 1–14 were first built as standalone applications in `Practical-N/` folders, several of them with their own database. Practicals 5–14 were then integrated one by one into the portal `studenthub/`. Until this step, `Practical-15/` was an older, independently kept copy of the portal with its own `schema.sql` (TEST_REPORT §6 noted that it was "still the older copy of the portal"). Practical 15 closes the project:

- One application (`studenthub/`), one database (`studenthub_db`) and one login for every feature of Practicals 5–14.
- `Practical-15/` becomes an exact, verified snapshot of `studenthub/` instead of a separately maintained copy.
- Installation, configuration, account creation and deployment are documented (`README.md`, `DEPLOYMENT.md`).
- Every test suite runs on disposable databases in one complete run, and one continuous Microsoft Edge scenario walks through the critical workflows with screenshots.
- The remaining limitations are stated openly.

2. Tools and Technologies

Tool / technology	Use in the final portal
PHP 8.2.12 (XAMPP) with `mysqli`, `fileinfo`, `mbstring`	All pages, the JSON API and the business rules (README requirement: PHP 8.1+)
MariaDB 10.4.32	The database `studenthub_db` (README: MariaDB 10.4+ or MySQL 8)
HTML5, CSS3, JavaScript	Semantic pages, the design system `assets/css/global.css`, Fetch API front ends (`events-live.js`, `data.js`), real-time validation (`register-validation.js`)
Bootstrap 5.3.2, Bootstrap Icons 1.11.1 (CDN)	Layout of the Admin Console
Python 3, Playwright, Microsoft Edge (headless)	`tests/run_all.py`, the HTTP suites and the Edge suites
phpMyAdmin	Database screenshots of Practical 8 (`tests/browser_database_evidence.py`)
Git / GitHub	Version history; `tools/sync_practical15.py` copies the files tracked by Git
Windows 11, PHP built-in server	Test environment of 31 October 2026

3. Integrated Architecture

Every page of the portal lives in `studenthub/` and loads the same two foundations: `db.php` opens one MySQLi connection to the database named by the environment (default `studenthub_db`; there is **no** fallback database – if it is unreachable the portal answers HTTP 503 with a general message and logs the reason on the server), and `includes/auth.php` provides the one session, permission and CSRF system:

studenthub/includes/auth.php (lines 2–12) – the single authentication and authorisation layer

```
 2  // StudentHub – the one authentication and authorisation system of the portal (Practicals 9 and 10
    integrated).
 3  //
 4  //   start_secure_session()   session cookie STUDENTHUBSESSID: HttpOnly, SameSite=Strict, Secure on
    HTTPS, strict mode
 5  //   require_login()          logged in, not idle longer than the timeout, account still exists (re-
    read every request)
 6  //   require_permission($p)   require_login() + role check; otherwise HTTP 403 page and an audit-log
    entry
 7  //   optional_user()          for public pages: the user if a valid session exists, else null (never
    redirects)
 8  //   csrf_field() / require_valid_csrf()   token for every state-changing POST form
 9  //   login_user() / end_session()           session ID regeneration at login, complete removal at
    logout
10  //
11  // Session keys (unchanged from the existing portal): user_id, username, email, role, plus
    last_activity and csrf_token.
12  // Roles stay exactly admin, editor and student; permissions are listed explicitly below.
```

Layer	Files	Responsibility
Pages and front end	`*.php` pages, `sidebar.php`, `assets/css/`, `assets/js/`	HTML5 pages, forms, the grouped sidebar, Fetch API front ends
Access control	`includes/auth.php`, `403.php`	Session cookie STUDENTHUBSESSID, 30-minute inactivity timeout, roles admin / editor / student with explicit permissions, CSRF tokens, security headers, the shared HTTP 403 page
Business rules	`includes/registration.php`, `student_management.php`, `event_management.php`, `event_registration.php`, `dashboard.php`, `navigation.php`, `inquiries.php`; `api/events.php`	Validation, transactions, capacity rule, soft delete, statistics – used by the pages and by the JSON API alike
Data and audit	`db.php`, `audit_logger.php` → `studenthub_db`	One connection; one audit log for security and data events
Files	`poster.php`, `uploads/`	Event posters, stored under random names and delivered only to signed-in users

3.1 Folder and file map

Path	Content
`studenthub/`	The canonical application (56 files tracked by Git)
`Practical-15/`	Verified snapshot of `studenthub/` plus its own `README.md` and `SNAPSHOT.md`
`Practical-1/` … `Practical-14/`	The original standalone versions, kept as historical evidence
`database/`	`studenthub_demo.sql` (new installation), `migrations/001–005` with rollbacks, migration tests
`tests/`	HTTP and Microsoft Edge suites, `run_all.py`, saved outputs
`tools/`	`create_demo_user.php` (accounts), `sync_practical15.py` (snapshot)

evidence/	Genuine screenshots of every integration step
README.md, TEST_REPORT.md, DEPLOYMENT.md, index.php	Documentation; the root index.php redirects to the portal

The 56 application files (the list with SHA-256 hashes is Practical-15/SNAPSHOT.md):

Group	Files
Public and account pages	index.php (home portal), login.php, register.php, logout.php, contact.php, 403.php
Student pages	dashboard.php (student_dashboard.php has identical content), profile.php, courses.php, attendance.php, assignments.php, results.php, achievements.php, notifications.php (Campus Events), events_live.php (Event Finder), data_explorer.php, directory.php
Administration	admin_dashboard.php; students.php, events.php, users.php, audit_logs.php redirect to its tabs
API and files	api/events.php, poster.php, uploads/.htaccess, uploads/.gitkeep
Shared PHP	db.php, audit_logger.php, sidebar.php, student_profile.php, includes/auth.php, registration.php, student_management.php, event_management.php, event_registration.php, navigation.php, dashboard.php, inquiries.php
Front end	assets/css/ (tokens, global, home, auth, bootstrap-theme), assets/js/ (nav, theme, script, auth, register-validation, data, events-live), data/ (students, events, faqs JSON)
Older pages	insert.php (answers HTTP 410 “Feature unavailable”), test_connection.php (Practical 8 array demo)

4. Database

All modules share studenthub_db (InnoDB, utf8mb4_unicode_ci, signed INT(11) keys). A new installation imports database/studenthub_demo.sql, which already contains migrations 001–005, fictional demo rows and no accounts or password hashes. An existing database is backed up with mysqldump and receives the missing migrations in order; schema_migrations records which ones are applied. Every migration checks its preconditions first, is idempotent and has a .down.sql rollback (database/README.md).

Migration	Practical	Changes	Tests (complete run; live)
001	8	students.user_id (UNIQUE, FK ON DELETE SET NULL), semester check, events.event_time / capacity (+ check), table registrations (CASCADE FKs, UNIQUE student + event, status), fk_audit_logs_user	88; live 24 / 24
002	11	UNIQUE students.enrollment_no	22; live 4 / 4
003	12	event_categories + fk_events_category, events.status, events.updated_at, UNIQUE events.poster	27; live 5 / 5
004	14	menu_items.min_role student / editor / admin + integrated links, users.status, chk_users_role	18; live 5 / 5
005	7	Table inquiries (FK to users ON DELETE SET NULL, topic ENUM, message CHECK 10–2000 characters, status new / resolved)	14; live 4 / 4

Relationship	Rule
users 1 : 0..1 students (students.user_id	ON DELETE SET NULL – the record stays, only the link

UNIQUE)	goes
`students` 1 : n `registrations`	`ON DELETE CASCADE` (the portal never deletes students – soft delete only)
`events` 1 : n `registrations`, UNIQUE (`student_id`, `event_id`)	`ON DELETE CASCADE` (the portal refuses to delete an event with registrations)
`event_categories` 1 : n `events`	`ON UPDATE CASCADE`, `ON DELETE RESTRICT`
`users` 1 : n `audit_logs`; `users` 1 : n `inquiries`	`ON DELETE SET NULL` – entries and inquiries are kept
`menu_items`, `schema_migrations`	No relationships

The capacity rule is enforced in PHP by `register_student_for_event()` (`includes/event_registration.php`): in one transaction it locks the event row with `SELECT … FOR UPDATE`, counts the registrations that are not cancelled and inserts (or reactivates) a registration only if a seat is free. Six simultaneous requests for three seats give exactly three registrations (README); the JSON API uses the same function.

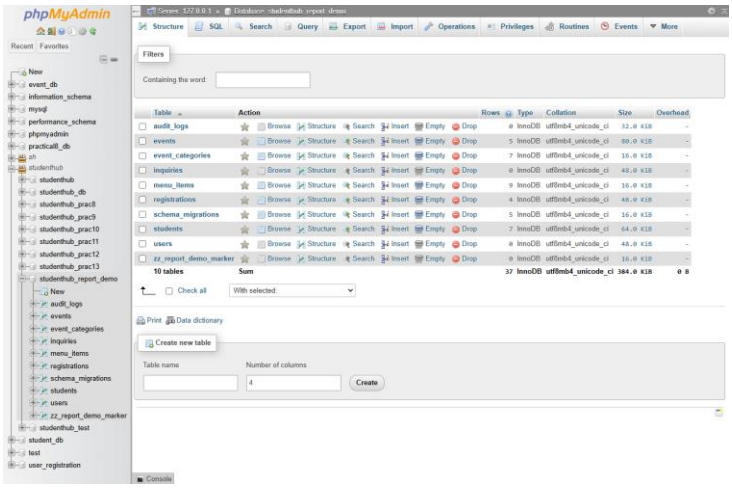


Figure 1: phpMyAdmin: the portal tables of a disposable copy built from studenthub_demo.sql (zz_report_demo_marker is the test's marker table) (01-database-tables.png)

5. How Each Practical Lives in the Portal

P	Feature in the portal	Files / route	Verified by (checks)	Commit
1	Planning: roles, sitemap, structure	documents in `Practical-1/` (nothing to integrate)	audit (TEST_REPORT §12)	–
2	Semantic HTML5 pages	all portal pages (header / nav / main / section / footer)	audit (TEST_REPORT §12)	–
3	Grid / Flexbox / responsive layout	`assets/css/global.css`, mobile drawer	audit (TEST_REPORT §12)	–
4	DOM: FAQ, modal, slider, banner, hamburger, theme	`index.php`, `assets/js/script.js`, `theme.js`	audit (TEST_REPORT §12)	–
5	Real-time regex validation, strength meter	`register.php`, `assets/js/register-validation.js`	`test_forms` 41 (P5–7), Edge 15	`938e74f`
6	Campus Data Explorer (Fetch, search, filter, sort, pages)	`data_explorer.php`, `assets/js/data.js`, `data/*.json`	as P5	`938e74f`
7	Stored Help & Contact inquiries	`contact.php`, `includes/inquiries.php`, `?tab=inquiries`, migration 005	as P5; migration 005: 14	`938e74f`
8	Database foundation, capacity rule in a transaction	migration 001, `includes/event_registration.php`	`test_001` 88	`37b35e6`
9	Secure self-registration (account + record in one transaction)	`register.php`, `includes/registration.php`	`test_auth` 132, Edge 14	`8011d39`

10	Login, sessions, timeout, POST logout, CSRF, roles	`login.php`, `logout.php`, `includes/auth.php`	as P9	`8011d39`
11	Student management, soft delete, CSV; inactive students cannot sign in	`?tab=students`, `includes/student_management.php`, migration 002	`test_students` 170, migration 002: 22, Edge 16 + 13	`0ac2f5d`, fix `6420efc`
12	Events with secure poster upload, registration, capacity, cancellation	`?tab=events`, `includes/event_management.php`, `poster.php`, `notifications.php`, migration 003	`test_events` 94, migration 003: 27, Edge 16	`ba11795`
13	Event Finder (Fetch API) + JSON API	`events_live.php`, `assets/js/events-live.js`, `api/events.php`	`test_api` 72, Edge 16	`a9d4e22`
14	Admin dashboard, role-based navigation, account status, audit filters	`admin_dashboard.php`, `includes/dashboard.php`, `navigation.php`, `sidebar.php`, migration 004	`test_dashboard` 36, migration 004: 18, Edge 15	`05927b3`
15	Final snapshot and end-to-end test	`Practical-15/`, `tools/sync_practical15.py`, `tests/browser_e2e.py`	snapshot 5, Edge end-to-end 17	see the repository history

The Admin Console routes are `admin_dashboard.php?tab=…`. Not carried into the portal: the canvas CAPTCHA of Practical 5 (a client-side CAPTCHA gives no real protection on a live form; the standalone `Practical-5/` keeps it). Practicals 11–14 are shown in the end-to-end scenario (Section 9); three practicals that the scenario does not touch are shown here:

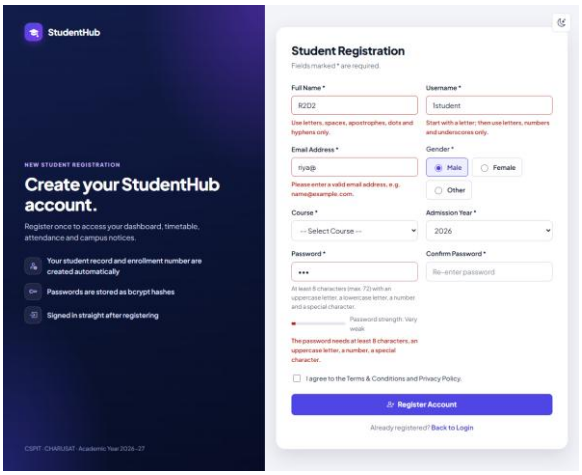


Figure 2: Practical 5 in the portal: messages and the strength meter appear while typing on register.php (01-register-live-validation.png)

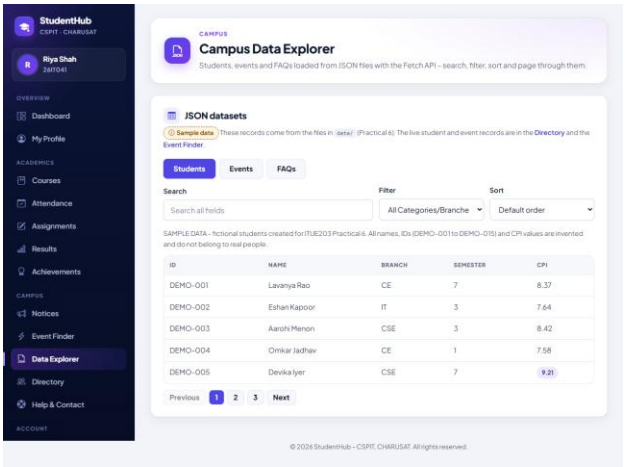


Figure 3: Practical 6 in the portal: the Campus Data Explorer renders data/students.json with search, filter, sort and pages (05-data-explorer-students.png)

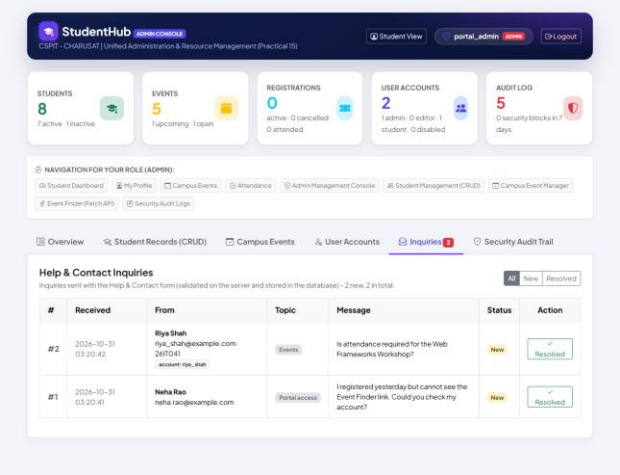


Figure 4: Practical 7 in the portal: stored inquiries in the Admin Console (Inquiries tab, new-count badge) (10-admin-inquiries.png)

6. Installation, Configuration, Accounts and Deployment

6.1 Local setup (Windows + XAMPP)

1. Requirements: XAMPP with PHP 8.1+ (tested 8.2.12) and MariaDB 10.4+ / MySQL 8 (tested 10.4.32); Python 3 only for the tests, Playwright + Microsoft Edge only for the browser tests. Start MySQL in the XAMPP Control Panel.
2. Create the database: `mysql -u root -e "CREATE DATABASE IF NOT EXISTS studenthub_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci"`, then `mysql -u root studenthub_db < database\studenthub_demo.sql`. An existing database: back it up and apply only the missing migrations (`SELECT * FROM schema_migrations` shows what is applied).
3. Create the first administrator: `php tools\create_demo_user.php --db=studenthub_db --username=admin01 --email=admin01@example.com --role=admin` – the password is prompted and stored only as a bcrypt hash.
4. Run `php -S localhost:8000 -t studenthub` and open `http://localhost:8000/` (or serve the repository root; its `index.php` redirects to `studenthub/`). The snapshot runs the same way: `php -S localhost:8015 -t Practical-15`.

6.2 Configuration and accounts

Environment variable	Default	Meaning
DB_HOST, DB_USER, DB_PASS, DB_NAME	localhost, root, (empty), studenthub_db	Database connection – there is no fallback database
STUDENTHUB_SESSION_TIMEOUT	1800	Inactivity timeout in seconds
STUDENTHUB_TIMEZONE	Asia/Kolkata	Time zone for dates (should match the database server)
STUDENTHUB_UPLOAD_DIR	studenthub/uploads	Folder for event posters (must not be reachable over HTTP)

No account and no password is stored in the repository. Administrators and editors are created with `tools/create_demo_user.php` (`--role=admin`, `--role=editor`, `--role=student`). Students normally register themselves at `register.php`, which creates the account (role always `student`) and its linked student record with the next enrollment number in one transaction. Administrators can disable, enable and delete accounts in the Admin Console; the own account and the last active administrator are protected.

6.3 Deployment on Apache (DEPLOYMENT.md)

- PHP 8.1+ with `mysqli`, `fileinfo`, `mbstring`; `AllowOverride All` so that `studenthub/uploads/.htaccess` denies HTTP access to the poster folder and switches PHP off there; HTTPS (the session cookie becomes `Secure` automatically).
- `studenthub/` is the document root; `database/`, `tests/`, `tools/`, `Practical-*` and `evidence/` must not be served. The web server user may write only to the poster folder (ideally outside the document root via `STUDENTHUB_UPLOAD_DIR`). `display_errors = Off`, `log_errors = On`.
- A dedicated database user with only `SELECT, INSERT, UPDATE, DELETE` on `studenthub_db`; schema and migrations are run with an administrative account; connection settings as `SetEnv` variables, never in a file inside the document root.
- Operation: regular `mysqldump --single-transaction` backups outside the web root (they contain password hashes), review of the audit log and the PHP error log, and before an update: back up, apply new migrations, run `python tests/run_all.py` against a copy, then deploy.

These steps were written for the project but were **not** carried out on a real public server; the application was tested with PHP's built-in server.

7. The Practical 15 Snapshot

`Practical-15/` is not maintained by hand. `python tools/sync_practical15.py` copies every file tracked by Git in `studenthub/` (`git ls-files`) byte for byte, removes application files that no longer exist in `studenthub/` – the stale files of the old copy, such as its own `schema.sql`, were removed this way and remain in the Git history – and writes `Practical-15/SNAPSHOT.md` with the SHA-256 hash of every file and the base commit. Uploaded posters are never copied; `README.md` and `SNAPSHOT.md` are the only files of the folder that are not application files. `python tools/sync_practical15.py --check` only verifies and exits with code 1 if anything differs:

tools/sync_practical15.py (lines 26–39) – byte comparison and detection of extra files

```
 26  def differences(files):
 27      problems = []
 28      for rel in files:
 29          a, b = SRC / rel, DST / rel
 30          if not b.is_file():
 31              problems.append(f"missing in Practical-15: {rel}")
 32          elif a.read_bytes() != b.read_bytes():
 33              problems.append(f"differs: {rel}")
 34      wanted = set(files) | OWN
 35      for p in DST.rglob("*"):
 36          rel = p.relative_to(DST).as_posix()
 37          if p.is_file() and rel not in wanted and not (rel.startswith("uploads/") and rel not in
     ("uploads/.htaccess", "uploads/.gitkeep")):
 38              problems.append(f"extra file in Practical-15: {rel}")
 39      return problems
```

The manifest lists 56 files and records as base commit `938e74f` (the Practicals 5–7 integration); the copied files may include changes committed together with the manifest. `tests/test_snapshot.py` checks the snapshot in five steps:

#	**Check in** `tests/test_snapshot_output.txt`	**Result**
1	`tools/sync_practical15.py --check`: `Practical-15/` is byte-for-byte identical to `studenthub/` and `SNAPSHOT.md` matches	PASS
2	`SNAPSHOT.md` lists all 56 tracked application files	PASS
3	The snapshot folder has no separate database or configuration of its own (no `schema.sql`, no config files)	PASS
4	`tests/test_auth.py` against `Practical-15/`: 132 passed, 0 failed	PASS
5	`tests/test_api.py` against `Practical-15/`: 72 passed, 0 failed	PASS

8. Security Summary

Area	Control in the portal	Verified by
Passwords	`password_hash()` (bcrypt); policy 8–72 characters with upper- and lower-case letter, digit and special character; no passwords or hashes in the repository	`test_auth`
Sign-in	One generic message; unknown accounts are checked against a dummy hash; session ID regenerated at login	`test_auth`, Edge
Sessions	Cookie `STUDENTHUBSESSID` (HttpOnly, SameSite=Strict, Secure on HTTPS, strict mode); 30-minute timeout; account re-read on every request; logout only by POST	`test_auth`
Access control	Explicit permissions per role; HTTP 403 page + `ACCESS_DENIED`; the API answers 401 / 403 in JSON	`test_dashboard`, `test_api`, end-to-end 4 and 17
Blocked accounts	Disabled accounts and student accounts with an inactive record cannot sign in; open sessions end (`SESSION_REVOKED`)	`test_students`, `test_dashboard`, end-to-end 6/7
CSRF	Token on every POST form; `X-CSRF-Token` header for the API; refusals logged as `CSRF_REJECTED`	`test_auth` and the module suites, end-to-end 17
SQL injection	Prepared statements for user input; `%` and `_` escaped in searches	`test_students`, `test_events`, `test_api`, `test_dashboard`
Cross-site scripting	`htmlspecialchars()` / `e()` on output; `textContent` in JavaScript; HTML escaped in JSON	`test_students`, `test_events`, `test_api`, `test_forms`, `test_dashboard`
File uploads	JPEG / PNG / WebP up to 2 MB; content type must match; real image; no embedded PHP; random 32-hex name; `uploads/.htaccess`; `poster.php` with `nosniff` and a sandbox CSP for signed-in users	`test_events`, end-to-end 8/9
Data rules	Capacity in a locking transaction; duplicates refused; soft delete of students; events with registrations cannot be deleted	`test_events`, `test_api`, `test_students`, end-to-end 10–12
Headers and errors	`X-Frame-Options: DENY`, `nosniff`, `Referrer-Policy`, `no-store` on private pages; errors only in the server log; HTTP 503 without details	`test_auth`
Audit and export	One filterable audit log; CSV cells starting with =, +, -, @, tab or CR are prefixed with an apostrophe	`test_dashboard`, `test_students`, end-to-end 16

9. End-to-End Scenario

`tests/browser_e2e.py` drives one continuous scenario in Microsoft Edge (1280 × 900 window, full-page screenshots) on the disposable database `studenthub_p11_test` and a temporary poster folder; the live database is never used. Five browser contexts play a new student (Ishaan Mehta), the administrator (`portal_admin`), an editor (`neha_editor`), a second student (Kunal Desai) and a visitor. Result: **17 / 17 passed** (`tests/browser_e2e_output.txt`).

Check	Workflow and assertion	Figure
1	Registration: account + linked student record with the next enrollment number, signed in	5
2	Logout (POST) and login with the e-mail address	6

3	The profile shows the student's own record	7
4	Editor and student opening the Admin Console → HTTP 403 “Access Denied”	8
5	Admin adds a student (automatic enrollment number) and edits the semester	9
6/7	Deactivation: the signed-in student loses access; a new sign-in is refused with the generic message	10
6	Reactivation restores sign-in	11
8/9	A PHP file disguised as `.jpg` is refused; a real JPEG poster is stored under a random name; the event is edited	12, 13
10/11	Registration succeeds; a second attempt → “already registered”; another student on a stale page → “The event is full.”	14, 15
12	Cancellation keeps the row as `cancelled` and frees the seat	16
13	Event Finder: debounced search and registration through the JSON API without a page reload	17
14	Student search + status filter	18
15	Dashboard counters equal the database	19
16	Audit log: denied access, refused sign-in, revoked session and rejected upload are recorded	20
17	A forged form without the CSRF token → “Security check failed” (403), nothing changed; the API answers 401 to a visitor	21
–	No JavaScript errors (only the intended 401 / 403 / 422 responses); no PHP warnings or errors in the server log	–

The run also reported the known 404 for `/favicon.ico` as information (the portal has no favicon).

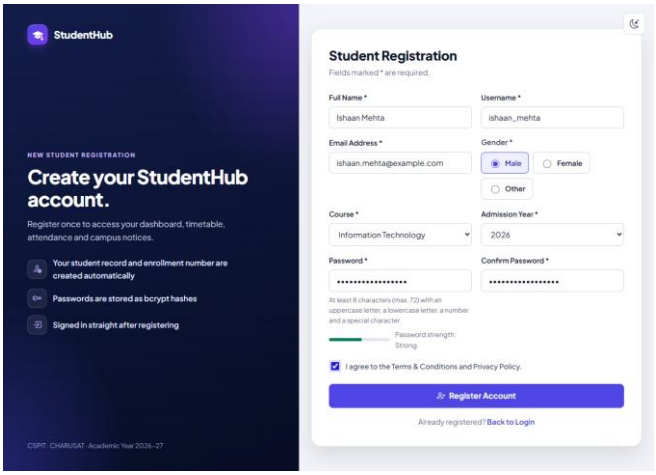


Figure 5: Step 1 – the registration form filled in for Ishaan Mehta (password strength “Strong”) (01-registration.png)

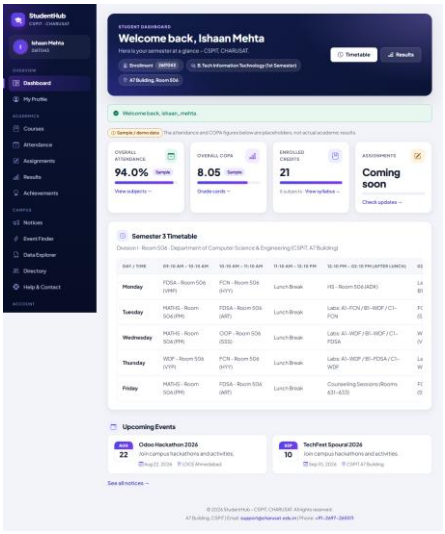


Figure 6: Step 2 – signed in again with the e-mail address: dashboard with enrollment number 26IT043 (02-login-dashboard.png)

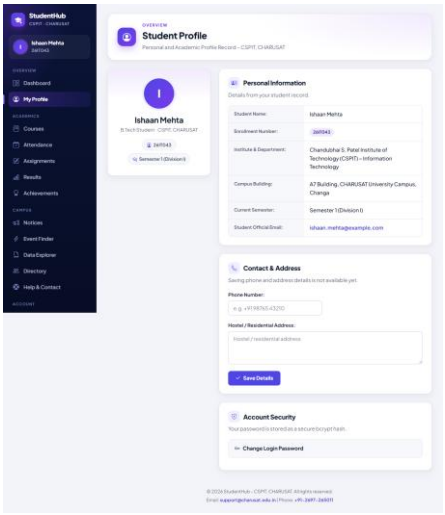


Figure 7: Step 3 – the profile shows the student's own record (03-student-profile.png)

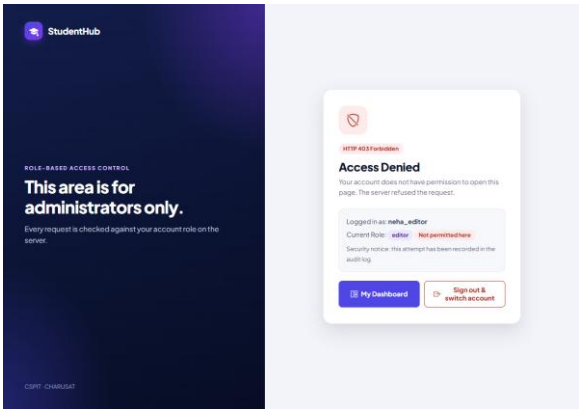


Figure 8: Step 4 – the editor opening the Admin Console gets the HTTP 403 page (04-editor-access-denied.png)

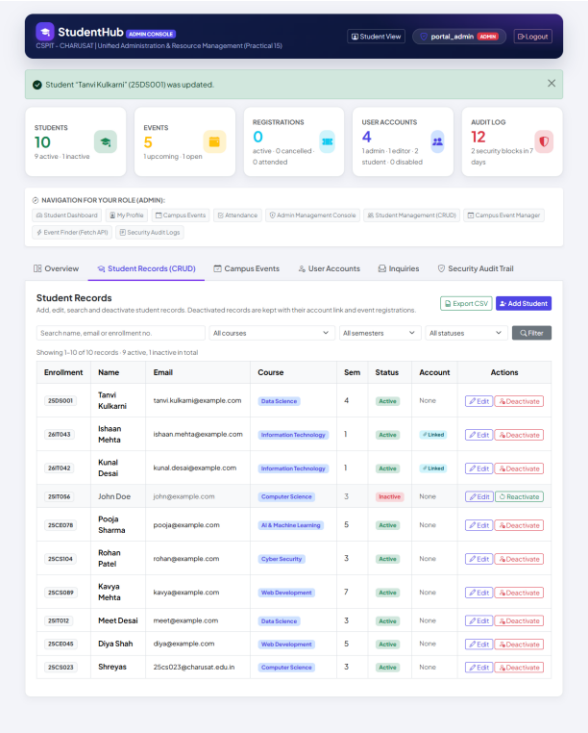


Figure 9: Step 5 – student "Tanvi Kulkarni" added with the automatic number 25DS001 and updated (semester 4) (05-student-created-edited.png)

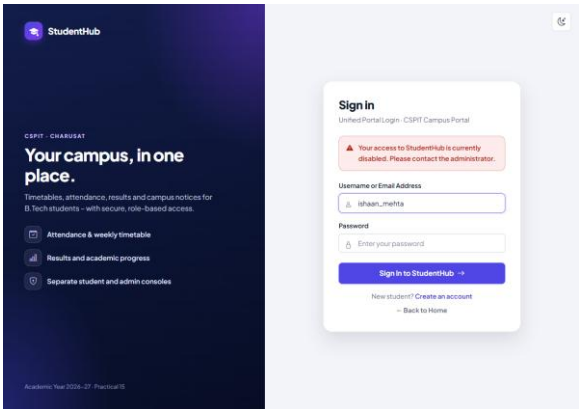


Figure 10: Steps 6/7 – after deactivation the student's sign-in is refused with the generic message (06-inactive-login-denied.png)

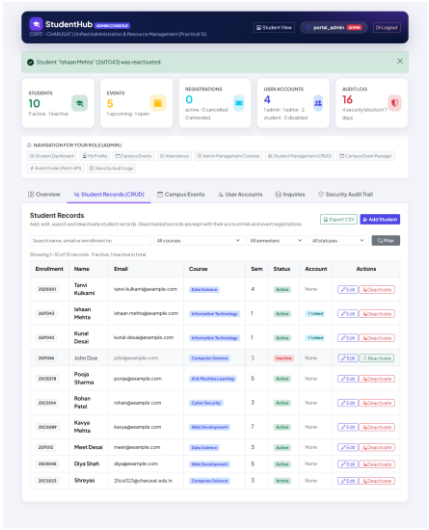


Figure 11: Step 6 – the record reactivated; sign-in works again (07-student-reactivated.png)

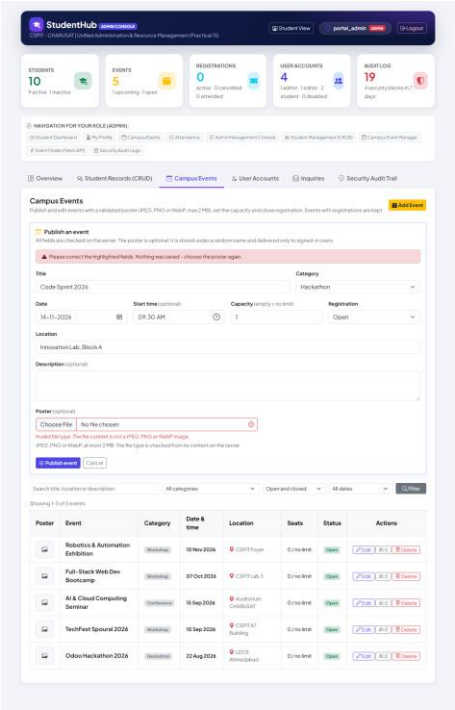


Figure 12: Step 8 – a PHP file named .jpg is refused: "The file content is not a JPEG, PNG or WebP image.", nothing saved (08-poster-rejected.png)

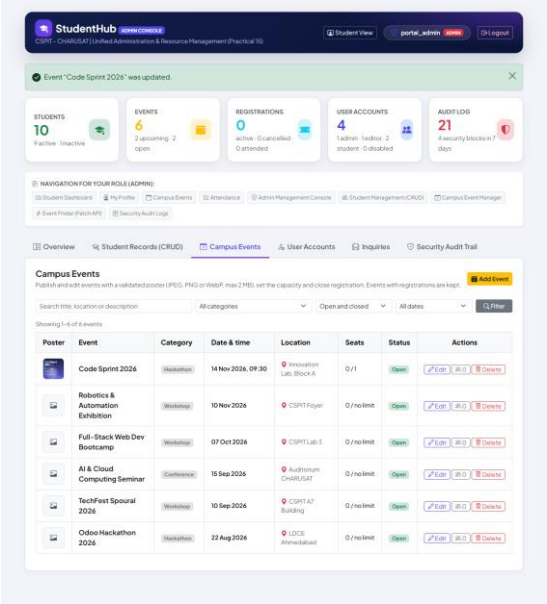


Figure 13: Step 9 – event "Code Sprint 2026" (capacity 1) with its stored poster, after editing (09-event-with-poster.png)

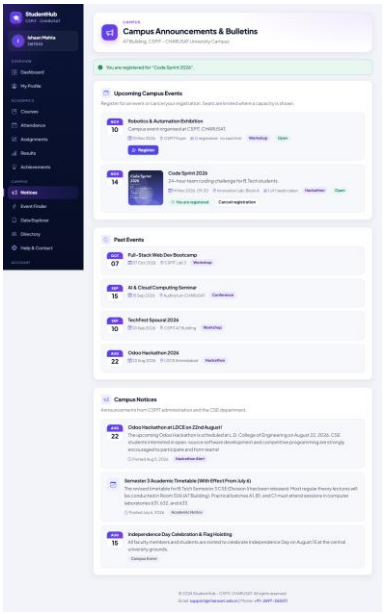


Figure 14: Step 10 – the student is registered for "Code Sprint 2026" (1 of 1 seats taken) (10-event-registration.png)

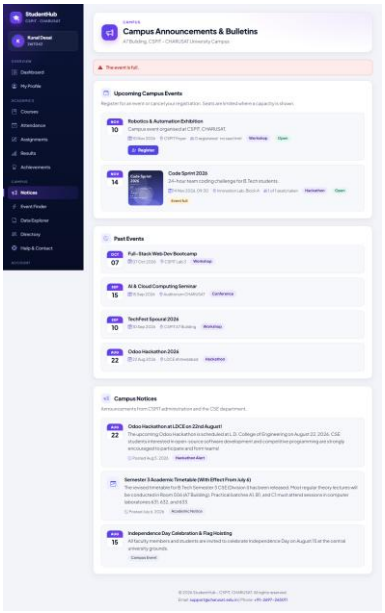


Figure 15: Step 11 – the second student gets "The event is full." (11-event-full.png)

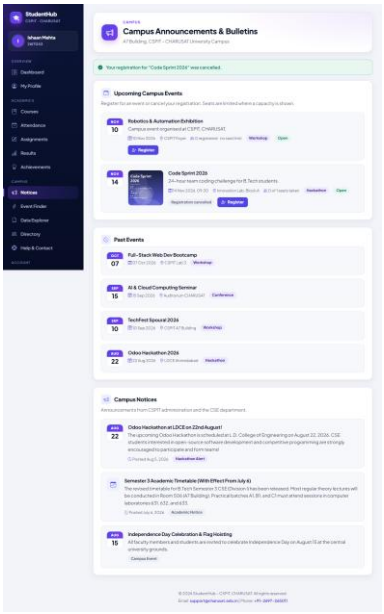


Figure 16: Step 12 – the registration is cancelled; the row is kept and the seat is free again (12-registration-cancelled.png)

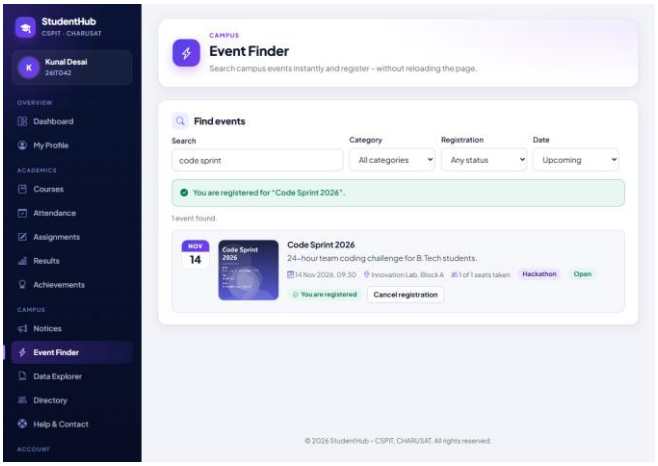


Figure 17: Step 13 – Event Finder: search "code sprint" and registration through the JSON API without a reload (13-fetch-api-registration.png)

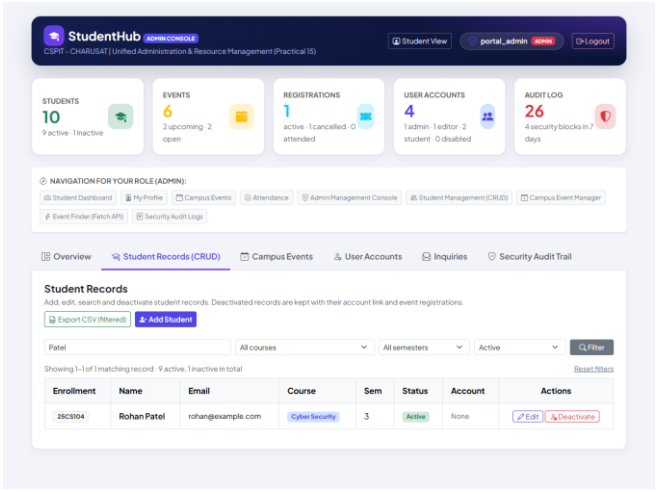


Figure 18: Step 14 – student search "Patel" combined with the status filter Active (14-search-filters.png)

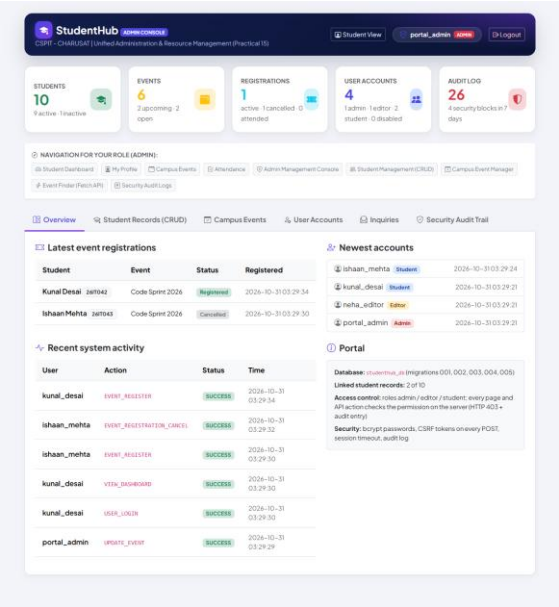


Figure 19: Step 15 – dashboard counters and the latest registrations (registered and cancelled) (15-dashboard-statistics.png)

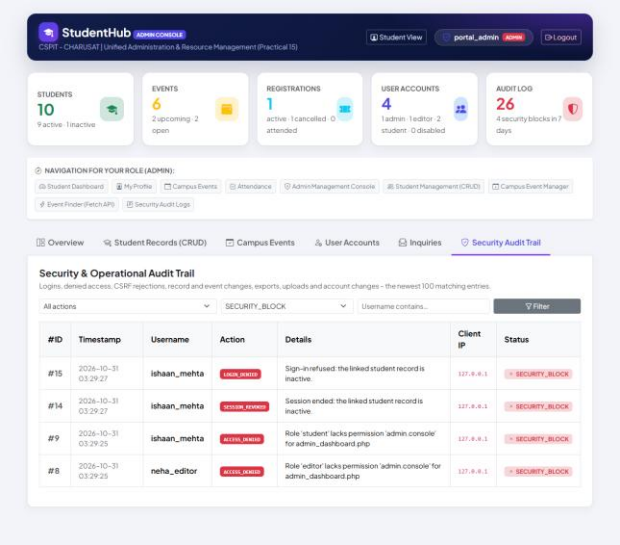


Figure 20: Step 16 – audit log filtered to SECURITY_BLOCK: refused sign-in, revoked session and two denied console accesses (the rejected upload has the status FAILED and is checked in the database) (16-audit-log.png)

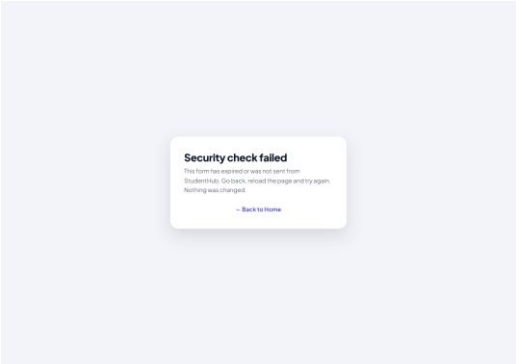


Figure 21: Step 17 – a forged form without the CSRF token: "Security check failed", nothing was changed (17-csrf-rejected.png)

10. Test Cases and Results

Complete run on 31 October 2026 – `python tests/run_all.py --browser <python with Playwright>` – with PHP 8.2.12 (XAMPP), MariaDB 10.4.32, Windows 11 and Microsoft Edge (headless, Playwright). Every suite uses its own disposable database (and a temporary poster folder); the live `studenthub_db` is only read. Output: `tests/run_all_output.txt`.

Suite	Passed	Failed	Time
Practical 8 – migration 001 and portal	88	0	13 s
Practical 11 – migration 002	22	0	7 s
Practical 12 – migration 003	27	0	6 s
Practical 14 – migration 004	18	0	5 s
Practical 7 – migration 005	14	0	4 s
Practicals 9/10 – registration, login, sessions, RBAC	132	0	22 s
Practical 11 – student management	170	0	16 s
Practical 12 – events, posters, registration	94	0	14 s
Practical 13 – JSON API, Event Finder	72	0	10 s
Practical 14 – admin dashboard	36	0	7 s
Practicals 5–7 – validation, Data Explorer, inquiries	41	0	5 s
Practical 15 – snapshot (incl. 132 + 72 checks against the snapshot)	5	0	33 s
Edge – registration, login, RBAC	14	0	5 s
Edge – student management	16	0	7 s
Edge – inactive student login	13	0	6 s
Edge – event management	16	0	11 s
Edge – Event Finder (Fetch API)	16	0	13 s
Edge – admin dashboard	15	0	9 s
Edge – Practicals 5–7	15	0	7 s
Edge – end-to-end (Practical 15)	17	0	13 s
Total (20 suites)	**841**	**0**	

12 database and HTTP suites contribute 719 checks and 8 Microsoft Edge suites 122 checks. `run_all.py` reads each suite's own last summary line and counts a suite that exits with an error as failed.

Live database: migrations 001–005 were applied to the working `studenthub_db`, each after a verified backup; the read-only live checks pass (001 24 / 24, 002 4 / 4, 003 5 / 5, 004 5 / 5, 005 4 / 4). **Database evidence for Practical 8:** `tests/browser_database_evidence.py` 6 / 6 (phpMyAdmin screenshots of a disposable copy).

11. Problems Encountered and Fixes

Problem	Fix
`Practical-15/` was an older, independently kept copy of the portal (with its own `schema.sql`) that no longer matched `studenthub/`	Replaced by the generated snapshot; the old copy stays in the Git history; `tests/test_snapshot.py` detects any difference
The first complete run showed one failure in the Event Finder browser test: the check for “Event full” ran before the list refresh after the API's 409 answer had finished (a timing race in the test; it had passed in every earlier run)	The test now waits for the refreshed card; the second complete run passed completely
Summary of the complete run	`run_all.py` was corrected to read each suite's own last summary line (TEST_REPORT §13)
Earlier integration steps (TEST_REPORT §7–§12): PHP used	Fixed in their steps (`Asia/Kolkata`, sign-in and

XAMPP's `Europe/Berlin` time zone; deactivated students could still sign in; the Event Finder's loading indicator stayed visible (`display: flex` overrode `hidden`); sidebar links from `menu_items` were not escaped; `data.js` inserted values unescaped and had an artificial 800 ms delay	session check, CSS fix, escaping, delay removed) – the complete run confirms all of them together

12. Known Limitations

- No limit on repeated sign-in attempts; no e-mail (password reset, answers to inquiries).
- Posters are uploaded only in the Admin Console (the JSON API handles no files); `menu_items` has no editing form.
- The Admin Console loads Bootstrap from a CDN; the portal has no favicon (one 404 per browser session).
- The canvas CAPTCHA of Practical 5 was not carried into the portal.
- Academic figures on the student pages (CGPA, SGPA, attendance, timetable) are sample data, labelled "Sample / demo data".
- The standalone `Practical-1/` … `Practical-14/` folders are historical and not updated.
- The deployment steps of `DEPLOYMENT.md` were not carried out on a public server.

13. Conclusion

StudentHub is now one application with one database and one login: the validation of Practical 5, the Data Explorer of Practical 6, the inquiries of Practical 7, the database foundation of Practical 8, the registration, login and sessions of Practicals 9 and 10, the student management of Practical 11, the events and posters of Practical 12, the Fetch API of Practical 13 and the admin dashboard of Practical 14 all run in `studenthub/` on `studenthub_db` through migrations 001–005. `Practical-15/` is a generated, hash-verified snapshot of exactly this application. The complete run passed all 841 checks of 20 suites, the snapshot itself passed the authentication and API suites, and the end-to-end scenario carried students, an editor and the administrator through 17 critical workflows in Microsoft Edge with genuine screenshots. The remaining limitations are listed above.

14. References

- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- PHP Manual – password_hash: https://www.php.net/manual/en/function.password-hash.php
- MDN – Using the Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- OWASP – Session Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- OWASP – File Upload Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- MariaDB Knowledge Base – SELECT … FOR UPDATE: https://mariadb.com/kb/en/for-update/
- Apache HTTP Server – .htaccess files: https://httpd.apache.org/docs/2.4/howto/htaccess.html
- Playwright for Python: https://playwright.dev/python/
- Project files: `README.md`, `DEPLOYMENT.md`, `TEST_REPORT.md` (section 13), `database/README.md`, `Practical-15/README.md`, `Practical-15/SNAPSHOT.md`
- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023